Генерация кода маленьким Transformer'ом

Владимир Родин
ДЗ2 по NLP, НИУ ВШЭ, 2026

Вступление + описание итоговой архитектуры

Вступление. Когда я узнал, что у нас будет домашнее задание с лидербордом и с использованием агентов, я очень обрадовался, ведь когда есть лидерборд лично у меня просыпается соревновательный дух, а разрешение на использование агентов помогает попробовать много разного и в итоге найти оптимальное решение. Мое решение этого дз скорее не про супер инновационное решение, а про грамотную работу с агентами, как мне кажется в современном мире это иногда даже важнее каких-то теоретических знаний в области. В первом разделе я расскажу про итоговую архитектуру решения, а затем будет долгий рассказ в хронологическом порядке как мы к этому пришли.

Задача. Обучить с нуля encoder-decoder Transformer, который по текстовой задаче из датасета TinyGSM пишет функцию `solve()` на Python. Ответ засчитывается, если код при исполнении выдаёт правильное число. Все 1919 тестовых задач нужно решить не дольше чем за 180 секунд на одной видеокарте T4.

Результат. 80,10% на публичном лидерборде (1–2 место на момент 21ое октября). Финальное решение — ансамбль из трёх моделей (≈100M + 54M + 54M параметров), который голосует по результатам исполнения кода и укладывается в ≈123 с.

Код и файлы. Весь код, ноутбуки и записи экспериментов лежат в публичном репозитории https://github.com/thisariv/hse-nlp-hw2-code-generation. Ниже пути указаны относительно корня репозитория. Главное для проверки: `submission/inference.ipynb` (финальный ноутбук), `submission/README.md` (как запустить) и `submission/code/hw2_codegen/` (чистый код инференса и обучения).

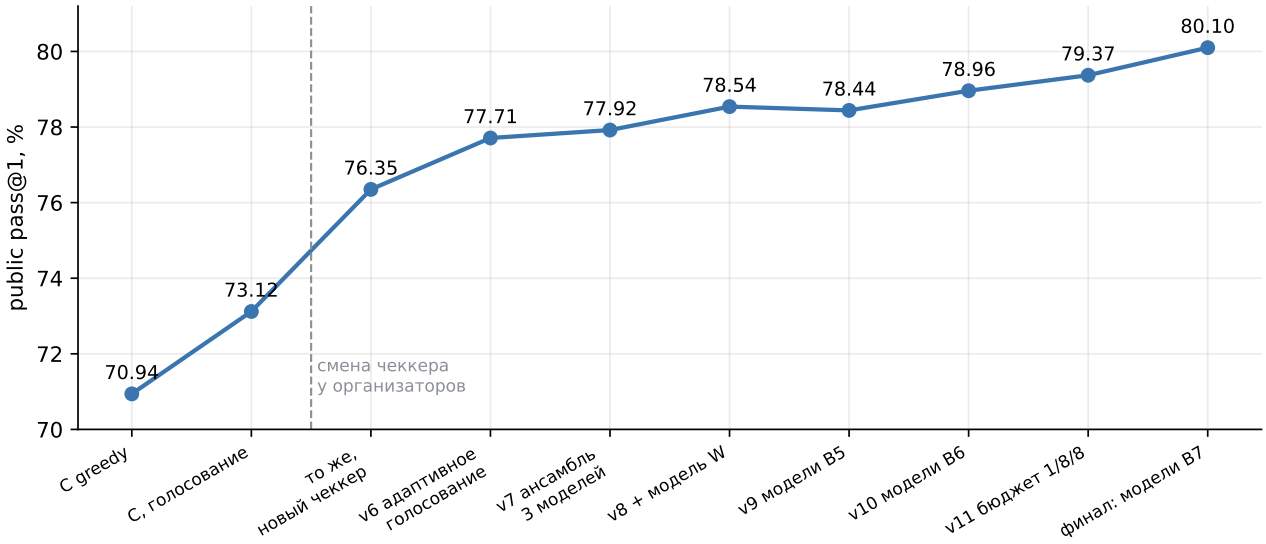


Рис. 1: Публичный лидерборд по нашим посылкам. Каждая точка — одно улучшение инференса или моделей.

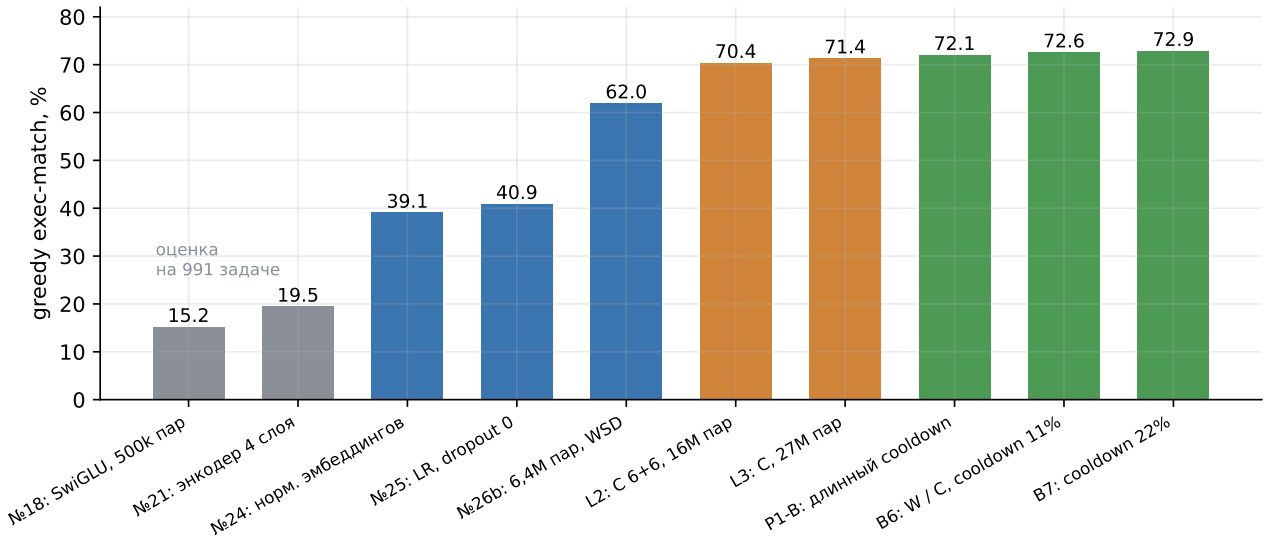


Рис. 2: Как росло качество одной модели (greedy, без голосования): от эксперимента №18 этапа 1 до финальных моделей этапа 2.

Описание итоговой архитектуры

Решение состоит из четырёх частей: данные и их представление, три модели, рецепт обучения и инференс с голосованием.

Данные. Используется только разрешённый датасет `SlavaYus/tinysms_qa_12m`. Мы отбросили пары, у которых вопрос длиннее 512 байт или код не короче 512 байт в UTF-8. Ещё убрали пары, чей эталонный код при исполнении не даёт конечного числа, и вопросы, совпадающие с тестом. Хешем (seed 42) отложили 4096 задач для валидации (*heldout4k*); её делим пополам на *tune* и *eval* по 2048. Осталось **9 725 737 пар** для обучения; текстовые дубли не удаляли. Новых данных и новых таргетов не создавали: меняется только запись того же кода.

Представление кода. Почти все программы начинаются с одинаковой «шапки» и заканчиваются строками `result = …` / `return result`. Мы убираем эти повторяющиеся куски обратимым преобразованием. Модель генерирует только смысловую часть, а перед исполнением код восстанавливается.

Исходный код

```
def solve() -> int:
    perimeter = 64
    side = perimeter / 4
    result = side
    return result
```

Что генерирует модель

```
<|header_tail|>
    perimeter = 64
    side = perimeter / 4
    side
```

Токенизатор. Собственный BPE на 8189 токенов в стиле Qwen: каждая цифра — отдельный токен, поэтому числа всегда разбиваются одинаково. Ещё 3 служебных токена режима записи и PAD/BOS/EOS — всего **8195** токенов. Ответ ограничен 512 токенами.

Модели. Все три модели — обычные encoder-decoder Transformer'ы (`nn.Transformer` из PyTorch) с одинаковой схемой слоя: Pre-LN (LayerNorm перед блоком), SwiGLU вместо обычного FFN, обученные позиционные эмбеддинги (до 2048 позиций), отдельная (не связанная с эмбеддингами) выходная голова, dropout 0. Эмбеддинги токенов инициализируются нормированными: веса делятся на $\sqrt{d}$, а на входе умножаются на $\sqrt{d}$ (см. эксперимент №24). Код — `submission/code/hw2_codegen/model.py`.

	C (C-long2x B7, P1-B)	**W** (W B7)
Слоёв энкодер + декодер	6 + 6	6 + 6
d_{model} / головы attention	512 / 8	768 / 12
Скрытый размер SwiGLU	1368	2048
QK-нормализация (LayerNorm на q и k в каждой голове)	нет	да
Параметров	≈54M	≈99M

Обучение. Общий рецепт для всех моделей (`submission/code/hw2_codegen/train.py`):

- AdamW (betas 0,9/0,999), weight decay 0,01, клиппинг градиента по норме 1,0;
- смешанная точность FP16 + GradScaler (начальный масштаб 1024);
- эффективный батч 128 пар (2 микробатча по 64); группировка по длине: примеры сортируются локально блоками по 4096 и режутся на батчи, порядок батчей перемешан;
- функция потерь — кросс-энтропия по токенам ответа; для W дополнительно z-loss с весом 10^{-4} (штраф за рост $\log \sum e^{\text{logit}}$, стабилизирует FP16);
- порядок данных *long2x*: длинные примеры встречаются в эпохе дважды (у C-long2x и W);
- расписание LR **WSD** (warmup–stable–decay): короткий линейный warmup, затем долгий *постоянный* LR, затем отдельный участок охлаждения до нуля по форме $\eta_j = \eta_0\big(1 - \sqrt{j/N}\big)$, где η_0 — постоянный LR, j — номер шага охлаждения, N — его длина.

Плюс WSD в том, что из любой точки постоянного участка можно сделать отдельное охлаждение, получить готовую модель и продолжить основное обучение дальше.

Модель	Warmup	Постоянный LR	Пар в основной части	Охлаждение
W B7	2000 шагов	$6 \cdot 10^{-4}$	19,8M	$1 - \sqrt{t}$, 33 000 шагов
C-long2x B7	1000 шагов	$1{,}5 \cdot 10^{-3}$	37,6M	$1 - \sqrt{t}$, 65 800 шагов
P1-B	1000 шагов	$1{,}5 \cdot 10^{-3}$	26,8M	$1 - \sqrt{t}$, 31 372 шага + loss truncation 10%

Loss truncation у P1-B: в каждом батче отбрасываются 10% примеров с самой большой ошибкой (отдельно по корзинам длины). Обучение шло сессиями по 10–11 часов на Kaggle (2×T4); между сессиями сохранялось полное состояние (веса, AdamW, GradScaler, RNG, позиция в данных), поэтому продолжение было бит-в-бит.

Инференс. Схема на рис. 3. Для каждой задачи:

1. **greedy** от модели-якоря W B7;
2. **этап 1**: по одному сэмплу ($T = 0{,}7$) от W B7, C-long2x B7 и P1-B;
3. каждый кандидат **исполняется** в песочнице (лимит 3 с, 768 МБ памяти, отдельный процесс); если все 4 числа совпали ($|a - b| \le 10^{-6}$), это ответ — так решаются **62%** задач;
4. иначе **этап 2**: досэмплируем до 8 сэмплов от C-long2x B7 и от P1-B (всего до 17 сэмплов плюс greedy);
5. **голосование**: кандидаты группируются по численному ответу, побеждает самая большая группа. При равенстве — группа, где есть greedy, затем — с наибольшим средним log-prob. Победивший код ещё раз исполняется для проверки.

Быстрый декодер. Обычная генерация батчем устроена так: берём N задач и генерируем ответы токен за токеном, пока не закончат *все*. Короткие ответы заканчиваются рано, но их строки остаются в батче, и GPU продолжает считать для них пустые токены (рис. 4, слева). Кроме того, после каждого шага программа спрашивает GPU «все ли закончили?» и ждёт ответа,

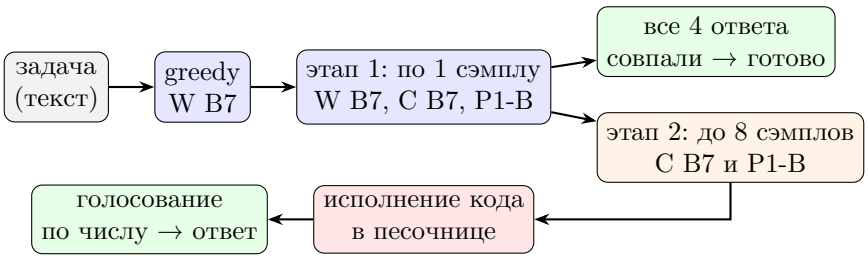


Рис. 3: Финальный инференс. Каждое решение исполняется; побеждает самый частый численный ответ.

а пока она ждёт, GPU простаивает. Для голосования нужно в ≈8 раз больше генераций, чем для greedy, поэтому такие потери становятся критичными. Наш декодер (`submission/code/hw2_codegen/decode.py`) устроен иначе:

- **KV-кеш**: ключи и значения уже сгенерированных токенов не пересчитываются (№07: в 6 раз быстрее);
- **батч по бюджету строк**: одновременно генерируется фиксированное число последовательностей (640), куда кладутся и разные задачи, и сэмплы одной задачи; задачи заранее отсортированы по длине вопроса;
- **уплотнение**: когда закончила четверть строк, законченные выкидываются, и батч становится меньше (рис. 4, справа);
- **проверка конца раз в 32 шага**, а не на каждом шаге, без ожидания GPU;
- **общий энкодер**: вопрос кодируется один раз, и ключи/значения cross-attention общие для всех сэмплов этой задачи;
- **сэмплирование одним действием на GPU**: к log-вероятностям добавляется случайный шум и берётся максимум (трюк «exponential race», эквивалентный обычному сэмплированию из softmax);
- **GPU и CPU работают одновременно**: пока GPU генерирует следующий батч, 4 процесса на CPU уже исполняют готовый код.

Профилирование и эти изменения ускорили greedy в 1,7–1,9 раза, а сэмплирование — в 1,3–1,6 раза (`work/notes/decode-speed.md`). В итоге голосование, которое в первой версии занимало 493 с (№27), стало укладываться в ≈120 с вместе с greedy и исполнением. Есть и защитный таймер: если к 160 с генерация не закончена, оставшимся задачам отдаётся greedy-ответ (в финальных прогонах он не срабатывал). Распределение времени — на рис. 5.

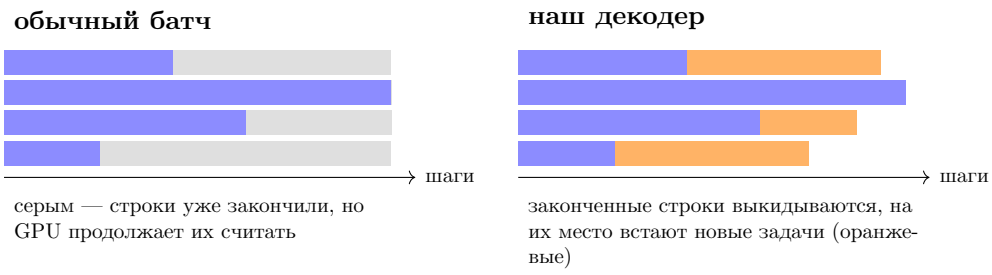


Рис. 4: Идея быстрого декодера: GPU не тратит время на уже законченные ответы.

Модель	Greedy, % (heldout4k)	Голосование 8 сэмплов (tune)
W B7 (якорь)	72,92	74,5
C-long2x B7	72,85	74,2
P1-B	72,07	74,3
Ансамбль (tune, 2048 задач, свежий прогон)		**76,2**

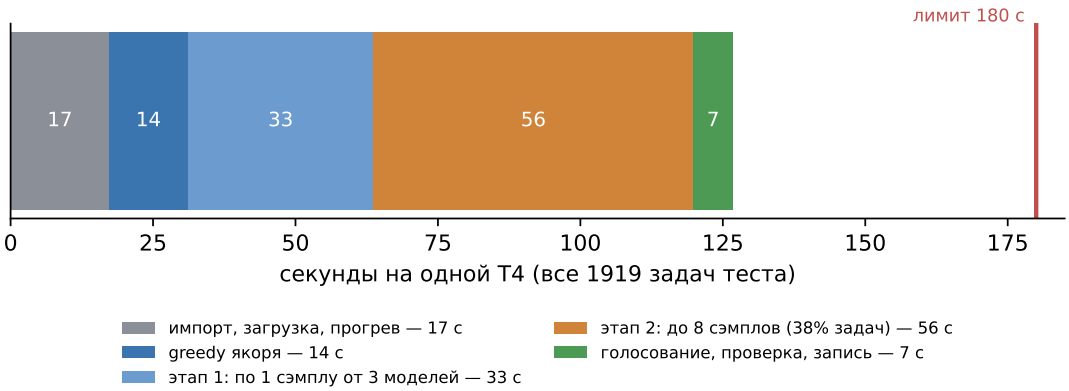


Рис. 5: На что уходит время на T4 (финальный прогон: 122,7 с, пик памяти 11,1 ГБ).

Воспроизводимость. Метод сэмплирует, а порядок досэмплирования на GPU зависит от тайминга. Поэтому два запуска дают немного разные ответы.

1. Этап 1

Теперь расскажу более подробно, что я делал. Я не хотел сразу обучать что-то большое с какими-то модификациями, поэтому я решил начать с самой базы, на это этапе я работал с codex gpt 6.1 sol max effort. Мы начали с самой простой модели: байтовый Transformer 2+2 слоя, $d = 128$, 1,25M параметров, 512 пар и 200 шагов обучения (`work/notebooks/01-baseline-byte-512-pairs-200-steps.ipynb`). Модель ещё не умела генерировать осмысленный код. Тогда мы проверили сам пайплайн: сможет ли модель хотя бы **запомнить 16 примеров**. Смогла — все 16 точно к шагу 2250 (ноутбуки 03–04). Значит, код обучения и генерации работает, и можно масштабироваться.

Дальше мы шли маленькими шагами: в каждом ноутбуке менялась одна вещь, а модель была маленькой, поэтому один эксперимент занимал минуты или десятки минут в Colab. Так за неделю получилось провести 26 экспериментов (табл. 1). Все ноутбуки лежат в `work/notebooks/`, их результаты с выводами — в `work/results/`, а разбор каждого шага — в `work/notes/`.

Что поняли на этом этапе. Сильнее всего помогли не архитектурные трюки, а «скучные» вещи: скорость обучения, нормированные эмбеддинги и большой объём данных.

2. Этап 2

В какой-то момент я понял, что пора масштабироваться и поэтому перешел на другую схему взаимодействия с агентами (я уже так работал над разными своими проектиками и такая схема мне нравилась). На этот раз я общался с claude opus 5.5 как с оркестратором, важно чтобы он не тратил лимиты и самое важное контекст на написание кода, копания в логах и тд, благодаря этому claude не компактил контекст во время каждой сессии, что регулярно происходит если работать с кодекс даже на самом маленьком проекте. Для того чтобы claude не засорял контекст всю грязную работу делает codex, для этого claude может запускать codex в терминале через «codex exec». Дальше мы использовали много сессий codex как агента который пишет код, ревьюит код, анализирует результаты, проводит ресерч и тд. Может показаться, что для этого нужны какие-то супер подписки, но это не так, у меня самые базовые 20$ подписки на клод и гпт и этого вполне хватает. Вот маленькая нейрослопная схема:

Таблица 1: Эксперименты этапа 1. Номер — префикс ноутбука в `work/notebooks/`.

№	Что меняли	Что получилось
01–02	байтовый Transformer 2+2, $d = 128$; $200 \to 2000$ шагов	код почти не компилируется (3/64 синтаксически верных)
03–04	запомнить 16 пар	16/16 точно к шагу 2250 — пайплайн рабочий
05–06	20k пар, 10k $\to$ 50k шагов	форма программы лучше, правильность слабая
07	KV-кеш при генерации	инференс $370 \to 61$ с (в 6 раз), ответы совпали побитно
08	BPE-токенизатор на 4k	синтаксически верных $57 \to 95\%$, ошибки отступов $754 \to 2$
09–12	ширина 256, cosine LR, 100k пар, warmup	ширина без других изменений ухудшила; данные и расписание помогли loss
13–15	Pre-LN, обучение до 100k шагов, SwiGLU	стабильнее и лучше: численные совпадения $104 \to 117 \to 123$ из 888
16	кеш BPE, AMP FP16 при батче 8	кеш -6% времени шага; FP16 на маленьком батче медленнее
17–18	500k пар	новая база: 15% задач с правильным ответом (151/991)
19–22	BPE 8k с раздельными цифрами, 4 слоя в декодере/энкодере	до 19% (193/991), различия на грани шума
23	батч 128, FP16, группировка по длине	обучение быстрее в **5,7 раза**; инференс $71 \to 16$ с
24	нормированная инициализация эмбеддингов	**20,0 $\to$ 39,1%** — самый большой скачок
25	LR 0,003, dropout 0	40,9%
26a–b	7M пар с проверенным исполнением, WSD	$46{,}1 \to 54{,}0 \to 62{,}0\%$ при росте данных до 6,4M пар

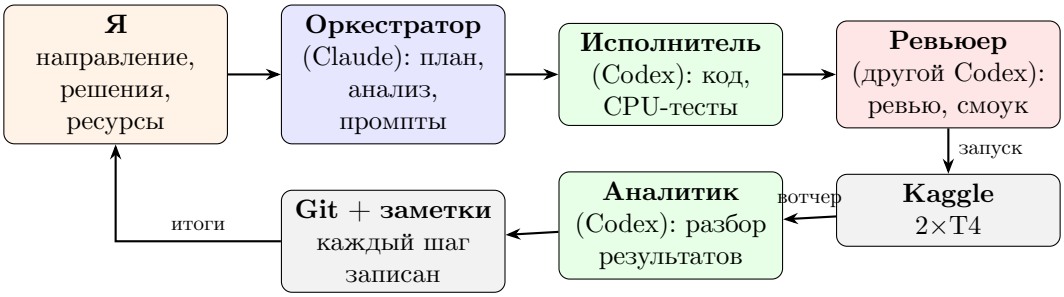


Рис. 6: Цикл одного эксперимента

Оркестратор и команда агентов. Обучение стало долгим (по 10 часов), и цена ошибки выросла. Поэтому работа разделилась на роли (рис. 6). Оркестратор пишет задание для Codex в виде промпта: контекст, что сделать, какие проверки пройти, что записать. Исполнитель готовит код и проверяет его на CPU. Другой, независимый Codex делает ревью и короткий смоук на GPU, и только после этого запускает долгую сессию. Когда сессия на Kaggle заканчивается, скрипт-вотчер скачивает результаты и запускает Codex-аналитика. Оркестратор читает короткий итог, коммитит и приносит мне решение.

Для начала я определил несколько правил которые надо было соблюдать, чтобы эксперименты не превратились в кашу и было понятно что к чему

- **Одно изменение за раз** и заранее записанная гипотеза.
- **Независимое ревью** перед каждым длинным запуском: код писал один агент, проверял другой. Затем **смоук** — короткий прогон на GPU на 10 минут.
- **Честная проверка**: выбираем вариант на одной выборке, подтверждаем на другой. Критерий успеха записываем *до* того, как смотрим на результат.

- **Всё в Git**: план, ревью, результаты и выводы каждого шага лежат рядом с кодом (work/notes/: *-plan.md, *-review.md, *-results.md).

Дальше — эксперименты этого этапа по порядку. Код запусков на Kaggle — в work/kaggle/ (сборщики build_*.py, исходники work/kaggle/src/), скрипты анализа — в work/research/.

2.1. Разведка и длинное обучение модели C

Сначала мы провели три короткие разведочные сессии на Kaggle (S1–S3, эксперименты №27–32):

- **S1 — голосование.** Голосование по 32 сэмплам дало 66,6% против 62,0% у greedy, но заняло 493 с вместо 180 (work/notes/27-execution-voting-s1-results.md). Стало понятно, что голосование — главный резерв, но нужен быстрый инференс.
- **S2 — ёмкость.** Модель побольше (B) при равном времени обучения оказалась не лучше базовой: 61,4% против 62,0% (work/notes/29m-28b-capacity-s2-results.md).
- **S3 — представление кода.** Компактная запись кода дала на 15% больше пар в секунду при том же качестве; её приняли как стандарт (work/notes/s3-representations-results.md).

Затем модель C (6+6, $d = 512$) учили длинными сессиями по 10 часов (work/notes/l1-long-training.md, l2-results.md, l3-results.md). В L1 обучили C с нуля (≈6M пар), в L2 и L3 продолжали две ветки из одной точки: обычный порядок данных (C-main) и порядок, где длинные примеры встречаются дважды (C-long2x). Каждые несколько часов мы делали короткое охлаждение (15 или 50 минут) и мерили качество (рис. 8). C-long2x оказалась немного лучше: после L3 **70,8%** (C-main) и **71,4%** (C-long2x) на 27M парах. Но рост почти остановился: последние 10 часов дали меньше 1 п. п.

Параллельно мы строили инференс. Первые посылки: одна модель greedy — 70,9%, голосование по 4+4 сэмпла двух моделей — 73,1%. Затем организаторы поправили чеккер, и та же посылка получила 76,4%. Дальше было несколько версий инференса (work/notes/final-inference.md):

- в v1 нашлась ошибка путей к данным Kaggle, в v2 — защитный таймер откатывал *все* ответы к greedy, а батчи считались по задачам, поэтому 2 сэмпла стоили столько же, сколько 4;
- v5 перешла на батчи по бюджету строк и стала сначала досэмплировать самые спорные задачи;
- v6 — адаптивное голосование (greedy, затем дополнительные сэмплы только при разногласии): **77,7%**;
- v7 — ансамбль трёх моделей C (C-main, C-long2x, P1-B): **77,9%**.

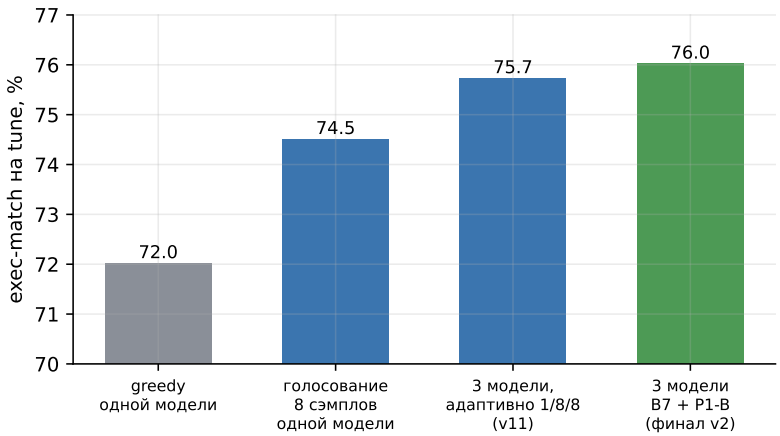


Рис. 7: Что даёт голосование (tune, 2048 задач): одна модель greedy → голосование 8 сэмплов → ансамбль трёх моделей.

2.2. Попытки пробить плато модели C

Мы проверили несколько гипотез, почему C перестала расти:

- **Переобучение?** Нет. На трейне и на отложенной выборке CE отличается всего на 0,047, точность по токенам — 93,6 и 92,4%, и разрыв не растёт (work/notes/overfit-check.md). Модель не запоминает трейн, а упирается во что-то другое.
- **Шум в разметке** (D1). Если в датасете есть пары с ошибочным эталоном, их можно найти по разногласию: несколько моделей решают задачу, и если все они уверенно расходятся с эталоном, эталон, скорее всего, неверный. Проверили это на повторяющихся вопросах, где правильный ответ известен по большинству: среди 886 кандидатов модели совпали с ним лишь в 14% случаев. Такой фильтр выкидывал бы в основном хорошие примеры, поэтому его не применили (work/notes/d1-results.md).
- **Глубже энкодер** (R1). Энкодер вырастили с 6 до 12 слоёв «с сохранением функции»: новые слои инициализировались так, что модель сначала выдаёт то же самое, а потом доучивается. За 10 часов — +0,15 п. п., в пределах шума (work/notes/r1-results.md).
- **Длинное охлаждение** (P1). Из конца L3 сделали две ветки: охлаждение 3,6 часа вместо 50 минут по форме $1 - \sqrt{t}$ (P1-A) и то же самое с loss truncation (P1-B). P1-A дала +0,3 п. п., P1-B — **+0,7 п. п.** (72,07%), так появилась модель P1-B (work/notes/p1-results.md). Оба прироста были на грани шума, и тогда мы ещё не поняли, насколько важно охлаждение.

2.3. Модель побольше: W с QK-нормализацией

По закону Chinchilla (Hoffmann et al., 2022) для фиксированного объёма вычислений есть оптимальный размер модели: слишком маленькая быстро «насыщается», слишком большая не успевает выучиться. По нашей оценке, модель C была мала: при нашем объёме обучения оптимум — ≈90–130M параметров (work/notes/architecture-decision.md). Мы замерили на T4 скорость обучения и генерации для 13 форм модели (разная глубина и ширина) и выбрали W (6+6, $d = 768$): сэмпл в 1,35 раза дороже, чем у C, и инференс ансамбля всё ещё укладывается в лимит.

B1: первая попытка провалилась. Две большие модели с нуля (W 6+6 и E 8+4, обе $d = 768$) учились 10,5 часа, но обучение периодически расходилось: loss резко рос, появлялись NaN. Автоматическое восстановление откатывало модель и дважды уменьшало LR, и в итоге обе модели отстали от C.

B2: QK-нормализация. Причина таких срывов в больших Transformer'ах — рост логитов attention: скалярные произведения запросов и ключей становятся огромными, softmax «залипает», а в FP16 случается переполнение. Мы добавили три вещи: LayerNorm на запросах и ключах в каждой голове (QK-norm), z-loss с весом 10^{-4} и warmup 2000 шагов. Обучение стало стабильным, и при одинаковом числе пар (3,1M) W обогнала C: 64,3% против 62,9% (work/notes/b2-results.md).

B3–B5: продолжение W. В B3 продолжили две ветки с разным LR: $6 \cdot 10^{-4}$ оказался лучше 10^{-3} (70,4 против 69,6% с 50-минутным охлаждением), дальше шли с ним. В B4 вторая ветка пробовала loss truncation в основном обучении — она хуже на 0,95 п. п. В B5 ещё 9 часов обучения W дали всего +0,05 п. п. (70,95% на 19,8M парах), а продолжение C-long2x до 37,6M пар — даже −0,24 п. п. При равном числе пар W была на 1,5–2 п. п. лучше C, но обе модели упёрлись примерно в 71%.

2.4. Главная находка: длинное охлаждение

Почему охлаждение вообще помогает? При большом постоянном LR веса модели «прыгают» по дну долины функции потерь и не опускаются в самую низкую точку. Охлаждение плавно уменьшает шаг, и модель «оседает». Наше плато оказалось ложным: модель продолжала копить знания на постоянном LR, но 50 минут охлаждения просто не хватало, чтобы это проявилось.

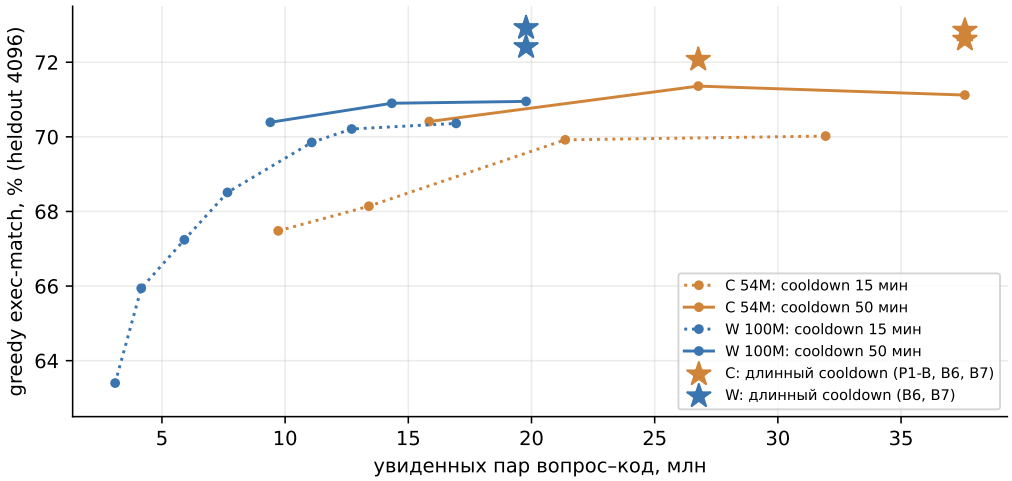


Рис. 8: Кривые обучения C и W. Точки — промежуточные модели с коротким охлаждением; звёзды — те же модели после длинного охлаждения.

В B6 мы взяли состояния W и C-long2x из конца основного обучения B5 и сделали длинное охлаждение формы $1 - \sqrt{t}$: 16 500 шагов для W и 32 900 для C, то есть ≈11% длины обучения, ≈4 часа (`work/notes/b6-results.md`). Это дало **+1,5 п. п. обеим моделям**: W 70,95 → 72,41%, C 71,12 → 72,61% (рис. 9). Обе модели впервые обогнали P1-B. В B7 охлаждение удвоили (≈22%, 7,4 часа): ещё +0,5 и +0,2 п. п., но это уже в пределах шума (`work/notes/b7-results.md`). Это совпадает с рекомендациями для WSD из литературы: охлаждение около 20% длины обучения, форма $1 - \sqrt{t}$ (Hägele et al., 2024).

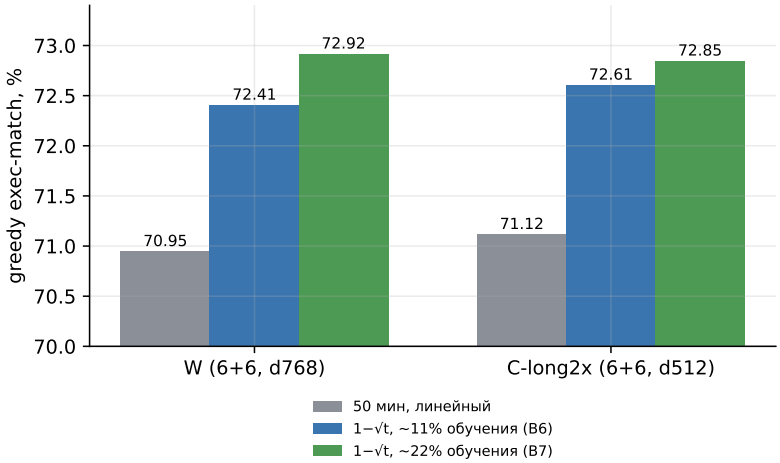


Рис. 9: Длина охлаждения. Те же стартовые веса (конец основного обучения B5), разный финальный участок.

2.5. Ансамбли и честная проверка

С каждой новой моделью мы обновляли ансамбль (рис. 1): v8 — добавили модель W (78,5%), v9 — модели после B5 (78,4%), v10 — модели после B6 (79,0%), v11 — больше сэмплов на спорных задачах (79,4%).

Отдельно мы проверили разные способы объединять модели (рис. 10). Для этого один раз собрали большой пул: по 16–24 сэмпла от каждой модели на 4096 задачах валидации, со всеми исполнениями. На этом пуле можно было бесплатно проверить любую политику: выбор — на одной половине (tune), подтверждение — на другой (eval) (`work/notes/e1-ensembles.md`, `e2-policy.md`, `tta-experiment.md`). Что пробовали, простыми словами:

- **ансамбль на уровне токенов**: три модели генерируют один ответ вместе, на каждом шаге

усредняя вероятности токенов; одна такая генерация хуже голосования по многим разным ответам ($-2{,}3$ п. п.);

- **перекрёстная оценка**: каждый кандидат оценивается всеми тремя моделями, и голос взвешивается этой оценкой ($+0{,}15$, шум);
- **model soup**: усреднить веса двух родственных моделей C в одну ($-0{,}05$);
- **смесь температур**: часть сэмплов при $T = 1{,}0$ ради разнообразия ($+0{,}7$, но на грани шума);
- **test-time augmentation**: подать вопрос в нескольких равнозначных формах (\$5 → 5 dollars), но такое правило меняет только 22% вопросов ($-0{,}05$);
- **больше сэмплов на спорных задачах** (1/8/8 вместо 1/5/6): **+0,73 п. п.** — единственное, что подтвердилось.

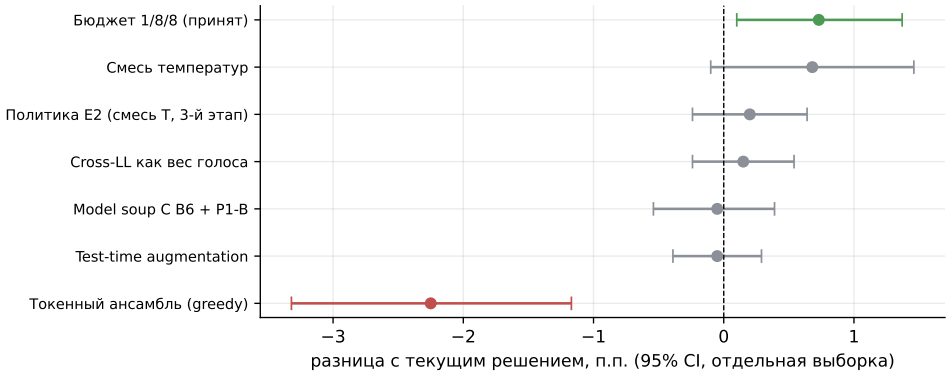


Рис. 10: Идеи для инференса, проверенные на отдельной выборке. Зелёный — прирост подтверждён, красный — вредит.

В конце мы перебрали все 120 троек из 10 сохранённых моделей. Оказалось, что десятки троек дают одинаковое качество в пределах шума, и P1-B ничем не особенна. Пять лучших вариантов мы отправили в бота (табл. 2, `work/notes/final-variants.md`). Все они оказались лучше v11 на public, а между собой в пределах шума. Финальным выбрали v2: у него лучший результат и на свежем tune, и на public, и он самый быстрый.

Таблица 2: Финальные варианты ансамбля. Первая модель — якорь (greedy).

	Состав	Время, с	Tune, из 2048	Public, %
v11	C B6 + P1-B + W B6	127	1547	79,37
v1	W B7 + C B7 + W B6	132	1552	80,00
v2 (финал)	**W B7 + C B7 + P1-B**	**111**	**1557**	**80,10**
v3	C B6 + P1-B + W B7	116	1552	79,90
v4	v11 + политика E2	151	1548	79,90
v5	C B6 + W B6 + C B7	111	1548	80,10

2.6. Что не сработало

Идея	Что произошло	Эффект
Loss truncation в основном обучении (B4)	отбрасывали 10% «трудных» примеров в каждом батче	−0,95 п. п.
Фильтр шумных пар (D1)	модели плохо находят ошибки в разметке: точность 14%	не применили
Глубокий энкодер 12 слоёв (R1)	рост «с сохранением функции»	+0,15 (шум)
Модель-ревьюер (код → вопрос)	переранжирование кандидатов	0
Продолжение обучения без длинного охлаждения (B5)	+9 ч обучения	+0,05 / −0,24
Test-time augmentation	варианты записи вопроса ($5 → 5 dollars)	−0,05
Токенный ансамбль, model soup, cross-LL	другие способы объединить модели	−2,25 ... +0,15
Greedy всех трёх моделей как голоса	голоса похожи и перевешивают сэмплы	−0,6 ... −0,9

2.7. Сколько вычислений ушло

Весь этап 2 шёл на Kaggle (2×T4). На рис. 11 все GPU-сессии этого этапа: длинные обучения по 4–11 часов, разведочные S1–S3 и около 40 коротких запусков (смоуки перед каждым длинным обучением, прогоны инференса, сбор пула E1, TTA, финальные варианты). Всего около 118 часов. Для L1 показано только основное обучение, а короткие запуски — оценка по сумме. Эксперименты этапа 1 шли в Colab на маленьких моделях и сюда не входят.

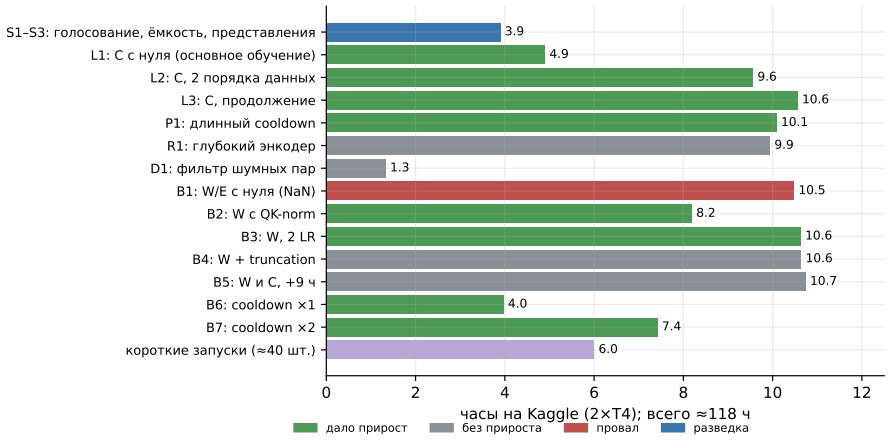


Рис. 11: GPU-сессии этапа 2 на Kaggle и их итог.

3. Выводы

- Маленькая модель с правильным рецептом решает ≈73% задач сама и ≈80% с голосованием по исполнению.
- Самые полезные решения — не архитектурные трюки, а «скучные» вещи: правильная метрика, скорость обучения, нормировка эмбеддингов, расписание LR и длинное охлаждение.
- Голосование быстро насыщается; дальше помогают только более сильные модели.

Где что лежит

Репозиторий: https://github.com/thisariv/hse-nlp-hw2-code-generation. Пути — относительно его корня.

Путь	Что там
submission/inference.ipynb	финальный ноутбук: генерирует ответы для теста на одной T4
submission/README.md	как запустить (Kaggle и Colab), веса, проверки
submission/code/hw2_codegen/	чистый код: модель, токенизатор, декодер, исполнение, голосование, обучение
submission/report/	этот отчёт (LaTeX и графики)
work/notebooks/01-*.ipynb ... 26b-*	эксперименты этапа 1
work/notes/	план, ревью и результаты каждого эксперимента
work/kaggle/	сборщики и код запусков этапа 2 на Kaggle
work/research/	скрипты анализа результатов
work/results/	итоговые метрики и таблицы (JSON, CSV, графики)
work/submissions/	все отправленные в бота файлы ответов

Веса финальных моделей — публичный Kaggle-датасет thisarivson/nlp-hw02-final-weights, версия 4: три FP16-файла весов, токенизатор, копия пакета `hw2_codegen` и входы теста. Как его подключить — в submission/README.md.